Assignment #04

Concurrent and Distributed Programming
a.y. 2025–2026

Buda Marco Merighi Daniele Turchi Jacopo

July 14, 2026

1 Distributed Smart Home Alarm System

The goal of this exercise is to extend the relevant part of the previous assignment to support execution on multiple cluster nodes, while also providing some level of fault tolerance to the alarm control unit.

1.1 Problem Analysis

Execution must begin by first creating the underlying cluster. Specifically, an instance of the actor system must be started for each node, while providing the necessary configuration. The configuration should be used to assign a unique address:port pair to each node, while also specifying at least one seed node, so that all nodes can automatically join into a single cluster.

The existing actors should then be initialized to run on the newly created cluster, ensuring that messages can always be sent regardless of physical location. For this reason, all message types will need to be updated to allow for serialization, which is required for actors running on different cluster nodes.

The central control unit should be further modified to restart automatically in case of failure, losing its current state.

1.2 Adopted Strategy

The proposed solution makes heavy use of Pekko's cluster sharding capabilities. The motivation for this decision is in the ability to reference domain actors in a way that is completely independent of their physical location. Additionally, it provides balanced distribution of entities across cluster nodes with no additional effort.

In this context, entities can be discovered with a call to `sharding.entityRefFor` by specifying simply their type key and identifier. Entity types with a single instance, such as the control unit, are given a fixed identifier.

The recovery behavior for the alarm control unit is implemented using supervision, by wrapping the initial behavior with a call to `Behaviors.supervise`. In case of failure,

the provided `RestartSupervisorStrategy` takes effect, restarting the underlying actor with its initial state ("safe mode").

The program features a showcase sequence that, compared to the previous assignment, includes a special `ForceFailure` message to verify the functioning of the recovery behavior. The message gets intercepted by the control unit and causes it to throw an exception, triggering the restart strategy.

2 Distributed TTT with Java RMI

This exercise asks for a distributed system that lets two remote players play Tic-Tac-Toe. A player can create a new game with a chosen name and wait for an opponent, or join an existing game given its name. The system is built around the distributed object computing model, using Java RMI as the underlying RPC mechanism.

2.1 Problem Analysis

From a concurrent and distributed point of view, the interesting part of the problem is not the game logic, which is trivial, but the management of *shared mutable state accessed by remote, independent and possibly faulty parties*. Three aspects are particularly relevant for the design:

- **Concurrent access to the game state.** RMI dispatches every incoming remote call on a thread taken from an internal pool, so calls from the two players (and the server's own background tasks) run on different threads at the same time. The board, the current turn and the game status form a single piece of state that must be kept consistent: without coordination, two moves could interleave and corrupt the board or let a player move out of turn.
- **Distributed notification (callbacks).** A move made by one player must be reflected on the other player's screen. Since neither client knows about the other, the server has to push updates back to the clients. This is naturally an *observer pattern realised over the network*: the client registers a remote callback object and the server invokes it. A remote callback is itself a blocking RPC that can be slow or fail, so it must not be performed while holding the lock on the game state.
- **Partial failure.** In a distributed system a peer can crash or disconnect silently, and an RPC can hang or throw. A player must not be left waiting forever for an opponent that will never move, so the system needs a way to detect unreachable peers and to react by aborting the game and releasing the associated remote resources.

2.2 Adopted Strategy

The code is organised in three packages. `common` holds the RMI interfaces (`GameLobby`, `Game`, `GameObserver`) and the data exchanged between the parties, `server` holds their

implementation, and `client` holds the interactive player application. The interfaces and the value objects are the only types shared by both sides.

Remote interfaces. `GameLobby` is the single entry point published in the RMI registry: it exposes `createGame`, `joinGame` and `listGames`. Both `createGame` and `joinGame` return a `Game` stub, the remote object the player uses to send its moves through `makeMove`. In the opposite direction, the client exports a `GameObserver` so that the server can call back `onStateChange` whenever the game evolves. All the data that crosses the network (`Board`, `Move`, `Mark`, `GameStatus` and the `GameSnapshot` that bundles them) are immutable, `Serializable` value objects, so they are passed *by value* as self-contained copies and cannot be a source of shared state between the two JVMs.

State consistency. Each game is a `GameImpl` instance that owns its board, turn and status. All the methods that read or change this state (`makeMove`, `join`, the abort and liveness logic) operate inside `synchronized` blocks on the game object, so the game behaves as a *monitor*: concurrent remote calls are serialised and the state always moves through legal transitions. Illegal actions (moving out of turn, moving on an occupied cell, moving before the opponent joined) are rejected with a `GameException`, which travels back to the offending client only. The `Board` is immutable: placing a mark returns a new board, which makes it safe to put a board inside a snapshot and hand it to another thread. The lobby itself keeps the running games in a `ConcurrentHashMap` and manipulates it only through its atomic operations (`putIfAbsent` to register a new game, `computeIfPresent` to attach the second player, `remove` to retire a finished game): creating, joining and listing games are therefore safe under concurrent calls without serialising every request on a single lobby-wide lock.

Decoupling notifications from the critical section. The state transition and the notification are decided together while holding the lock, but the actual remote callback must not run there, otherwise a slow or unreachable client would block the whole game. Each player therefore has a dedicated single-thread executor (the *notifier*). Inside the synchronized block we only *submit* the freshly computed snapshot to each player's notifier, and the slow RPC happens outside the lock. Using one thread per player also guarantees that snapshots are delivered to that player in the same order they were produced.

Per-player stubs. Instead of letting both players share one `Game` object, the lobby exports a small `PlayerProxy` per player, each bound to a fixed `Mark` (X for the creator, O for the joiner). When a proxy receives `makeMove`, it forwards the call together with its own mark, so a player can only ever play as itself and the server does not have to trust a mark sent over the wire.

Failure detection. `GameObserver` also declares a `ping` method. A shared scheduled executor periodically pings the observers of the active games. A player that misses several consecutive pings, or whose callback throws a `RemoteException`, is considered

gone. The game is then moved to the `ABORTED` status, the surviving player is notified, and the game terminates. A short RMI response timeout is set so that a broken connection fails fast instead of blocking the heartbeat. On the client side, `makeMove` is retried a few times before giving up, to tolerate transient network glitches.

Resource lifecycle. Every exported object reserves resources in the RMI runtime, so they must be released when no longer needed. When a game ends (by victory, draw or abort) its monitoring task is cancelled and its notifier threads are shut down, then the lobby removes the game from its registry and unexports both player proxies. The client likewise unexports its observer on exit. Unexport is forced, so an exported object is reclaimed even if a remote call is still in flight, and the lobby ignores the failure of a single unexport so that one stuck stub cannot prevent the rest of the game from being cleaned up.

Client application. The player application is a simple text client. Because the user types moves while updates may arrive from the server at any moment, a separate thread reads the keyboard and the main loop waits, with a classic `wait`/`notify` guarded by a lock, on whichever happens first between a new snapshot and a line of input. This keeps the screen reactive: if the opponent moves or disconnects while the player is being prompted, the client immediately re-renders the new state instead of staying stuck on the prompt.

3 Distributed Critical Sections with a Message-Oriented Middleware

3.1 Problem Analysis

The objective is to ensure mutually exclusive access to shared resources in a distributed environment without processes having explicit knowledge of one another. From a concurrency perspective, the main challenges are:

- **Race Conditions:** Simultaneous lock requests from distributed nodes must be serialized to prevent multiple processes from entering a critical section for the same resource.
- **Deadlocks and Faults:** A process crashing or disconnecting while holding a lock can halt the system permanently.
- **State Synchronization:** Lacking shared memory, the system relies entirely on asynchronous message passing, requiring precise correlation between requests and replies.
- **Starvation:** Queued requests must be managed to ensure fair access when resources become available.

3.2 Adopted Strategy

The solution implements a **Centralized Algorithm** for mutual exclusion utilizing a Message-Oriented Middleware (RabbitMQ). This approach guarantees that processes do not require explicit knowledge of one another, achieving **loose coupling**.

- **Architecture & Loose Coupling:** A centralized coordinator (the server) manages lock states and access permissions. Distributed processes act as clients, completely unaware of each other's existence, interacting solely with the MOM via a centralized request exchange.
- **Message Routing (Point-to-Point):** Clients publish `acquire` and `release` requests to a shared public queue. The server processes these and replies with grant messages to temporary, client-specific private queues, matching messages via correlation IDs to implement a response-based transient synchronous communication.
- **Concurrency Control & Fairness:** The server consumes messages using a single-thread executor. This guarantees a sequential processing of incoming requests, eliminating race conditions and ensuring fairness, as requests are handled in the order they are consumed by the broker.
- **Hierarchical Locks:** Resources are identified by path-based targets. The server evaluates conflicts by checking if paths are identical or if one is a direct ancestor or descendant.
- **Fault Tolerance & Liveness:** To prevent deadlocks caused by crashed or disconnected processes holding a lock, locks are granted with a 15-second lease duration. A scheduled timer evicts expired locks, ensuring the liveness property (every request is eventually granted) of the critical section.
- **Client Synchronization:** To bridge the asynchronous interaction model of the MOM with the blocking semantics expected of a lock acquisition, the client leverages `CompletableFuture` paired with a concurrent map. This blocks the client's execution thread until the asynchronous grant message is delivered or a timeout occurs.

3.3 Execution Instructions

To deploy and run the system, execute the following steps:

- **Start RabbitMQ:** Launch the message broker container using Docker:

```
docker run -it --rm --name rabbitmq \
  -p 5672:5672 -p 15672:15672 rabbitmq:4-management
```

- **Start the Server:** Execute the centralized lock server management application (`RabbitMQLockServer`) to listen for incoming requests.

- **Client Lock Utilization:** Distributed processes must instantiate and invoke the provided client interface (`DistributedLockManager`) to request and release locks on target resources.